Sécurisation du serveur SMTP : SPF, DKIM, DMARC, TLS et Fail2ban

Nov 8, 2026 · @arphan-rezilux

Auteur : Arphan Kaback Junior Bodian. Suite du guide technique « Messagerie interne d'entreprise sous Kali Linux (WSL) » du 7 novembre 2026.

Vue d'ensemble

Cinq mesures protègent le serveur : TLS chiffre les connexions, SPF, DKIM et DMARC empêchent l'usurpation du domaine, Fail2ban bloque les essais de mots de passe répétés. Seuls TLS et Fail2ban fonctionnent pleinement sur la maquette WSL ; les trois autres reposent sur le DNS public et se valident en laboratoire avec un petit serveur DNS local.

Mesure	Rôle	Outil	Laboratoire WSL	Production
TLS	Chiffrer et authentifier les connexions	Postfix, Dovecot, OpenSSL	Complet, avec une autorité interne	Let's Encrypt
SPF	Lister les serveurs autorisés à envoyer pour le domaine	Enregistrement DNS TXT, Rspamd	Via DNS local (dnsmasq)	DNS public
DKIM	Signer chaque message sortant	Rspamd	Signature réelle, clé publiée dans le DNS local	DNS public
DMARC	Dire quoi faire des messages qui échouent SPF et DKIM	Enregistrement DNS TXT, Rspamd	Via DNS local	DNS public
Fail2ban	Bannir les adresses qui échouent à répétition	fail2ban	Complet, sous réserve du pare-feu WSL	Complet

Hypothèses de ce rapport. Le domaine des comptes est `entreprise.sn` (celui créé dans PostfixAdmin) et le serveur s'appelle `mail.entreprise.local`. Dovecot est en version 2.4.4 : les paramètres `ssl_server_cert_file`, `ssl_min_protocol` et `auth_allow_cleartext` sont ceux de cette série. Les variables `$DBUSER` et `$DBPASS` ne sont pas utilisées ici.

Méthode. Avant chaque modification : `sudo cp fichier fichier.bak`. Après : `sudo postfix check` et `sudo doveconf -n > /dev/null && echo OK`, puis le test d'envoi `swaks` de l'étape 10 du guide. Une mesure à la fois.

1. TLS : chiffrer et authentifier les connexions

Le certificat auto-signé « snakeoil » du guide chiffre déjà les échanges, mais il oblige à cliquer « accepter l'exception » : un attaquant placé sur le réseau peut alors présenter son propre certificat sans que personne ne le remarque. La solution de laboratoire est une petite autorité de certification (AC) interne ; la solution de production est Let's Encrypt.

1.1 Créer l'AC interne et le certificat du serveur

```
sudo mkdir -p /etc/ssl/entreprise && cd /etc/ssl/entreprise
sudo openssl genrsa -out ca.key 4096
sudo openssl req -x509 -new -key ca.key -sha256 -days 3650 -subj
"/CN=Entreprise Local CA" -out ca.crt
sudo openssl genrsa -out mail.key 2048
sudo openssl req -new -key mail.key -subj "/CN=mail.entreprise.local" -out
mail.csr
echo "subjectAltName=DNS:mail.entreprise.local" | sudo tee san.ext >
/dev/null
sudo openssl x509 -req -in mail.csr -CA ca.crt -CAkey ca.key -CAcreateserial
-days 825 -sha256 -extfile san.ext -out mail.crt
sudo chmod 600 ca.key mail.key && sudo chmod 644 ca.crt mail.crt
sudo adduser postfix ssl-cert
sudo chgrp ssl-cert mail.key && sudo chmod 640 mail.key
```

Le fichier `ca.crt` (jamais `ca.key`) s'installe sur chaque poste : Windows, `certmgr.msc`, « Autorités de certification racines de confiance » ; Thunderbird, Paramètres, Certificats. Une fois la maquette terminée, rangez `ca.key` hors du serveur.

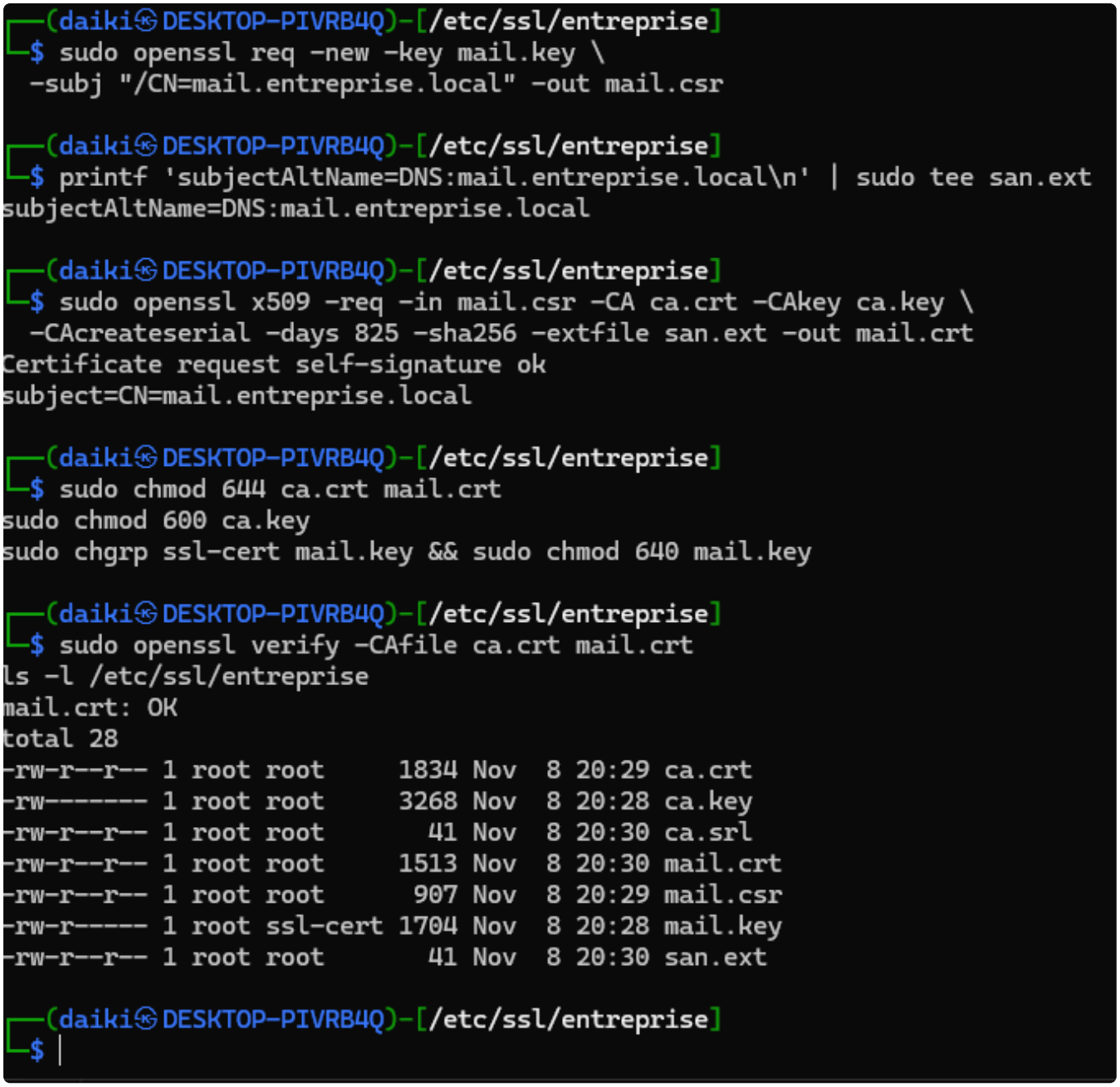


Figure 1. Création de l'autorité interne et du certificat du serveur ; la vérification renvoie OK.

1.2 Postfix : TLS imposé, versions récentes

```
sudo postconf -e "smtpd_tls_cert_file = /etc/ssl/entreprise/mail.crt"
sudo postconf -e "smtpd_tls_key_file = /etc/ssl/entreprise/mail.key"
sudo postconf -e "smtpd_tls_auth_only = yes"
sudo postconf -e "smtpd_tls_protocols = >=TLSv1.2"
sudo postconf -e "smtpd_tls_mandatory_protocols = >=TLSv1.2"
sudo postconf -e "smtp_tls_protocols = >=TLSv1.2"
sudo postconf -e "smtp_tls_mandatory_protocols = >=TLSv1.2"
sudo postconf -e "smtpd_tls_mandatory_ciphers = high"
sudo postconf -e "tls_ssl_options = NO_COMPRESSION"
sudo postconf -e "smtpd_tls_loglevel = 1"
```

La syntaxe `>=TLSv1.2` demande Postfix 3.6 ou plus (`postconf mail_version`). Le port 25 reste en TLS facultatif (`may`) : les autres serveurs de messagerie ne savent pas tous chiffrer, et un refus ferait perdre du courrier. Les ports des employés, eux, sont imposés.

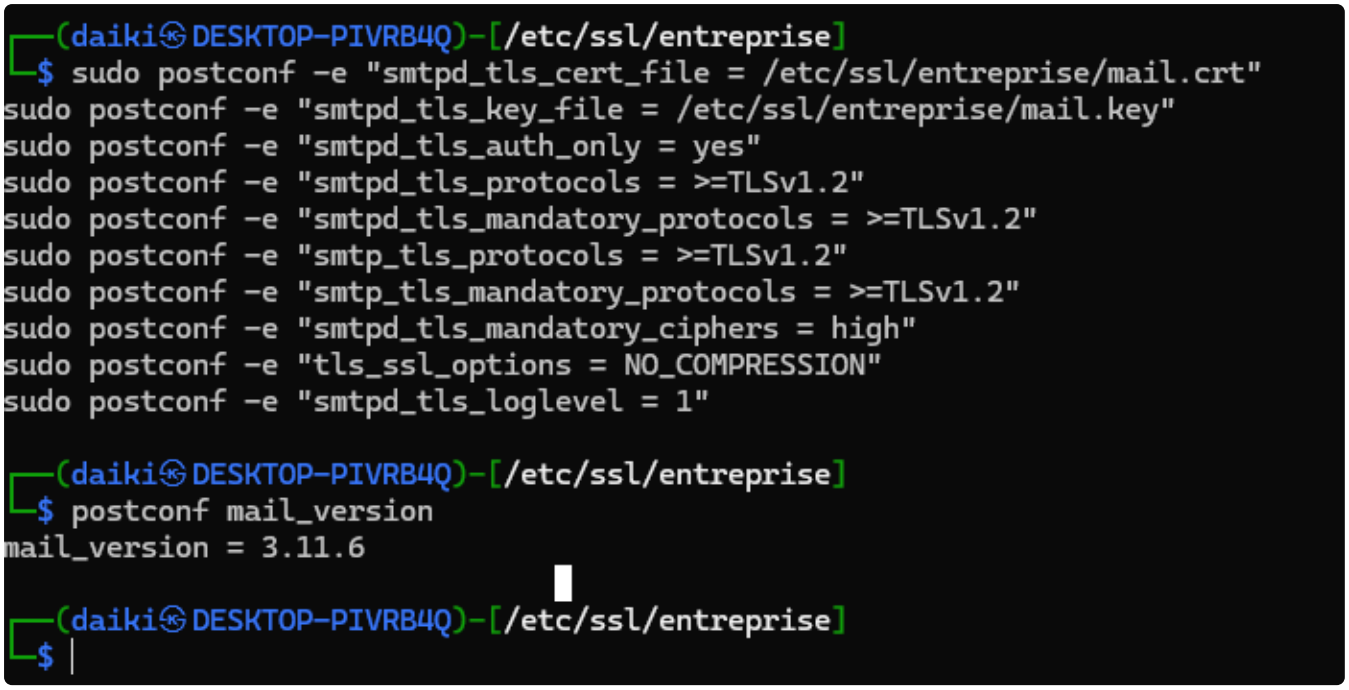


Figure 2. Paramètres TLS appliqués à Postfix (postconf, version 3.11.6).

Ajoutez le port 465 (TLS dès la connexion) à côté du 587 :

```
sudo postconf -M "submissions/inet=submissions inet n - n - - smtpd"
sudo postconf -P "submissions/inet/syslog_name=postfix/submissions"
sudo postconf -P "submissions/inet/smtpd_tls_wrappermode=yes"
sudo postconf -P "submissions/inet/smtpd_tls_security_level=encrypt"
sudo postconf -P "submissions/inet/smtpd_sasl_auth_enable=yes"
sudo postconf -P
"submissions/inet/smtpd_relay_restrictions=permit_sasl_authenticated,reject"
sudo postfix check && sudo systemctl restart postfix
```

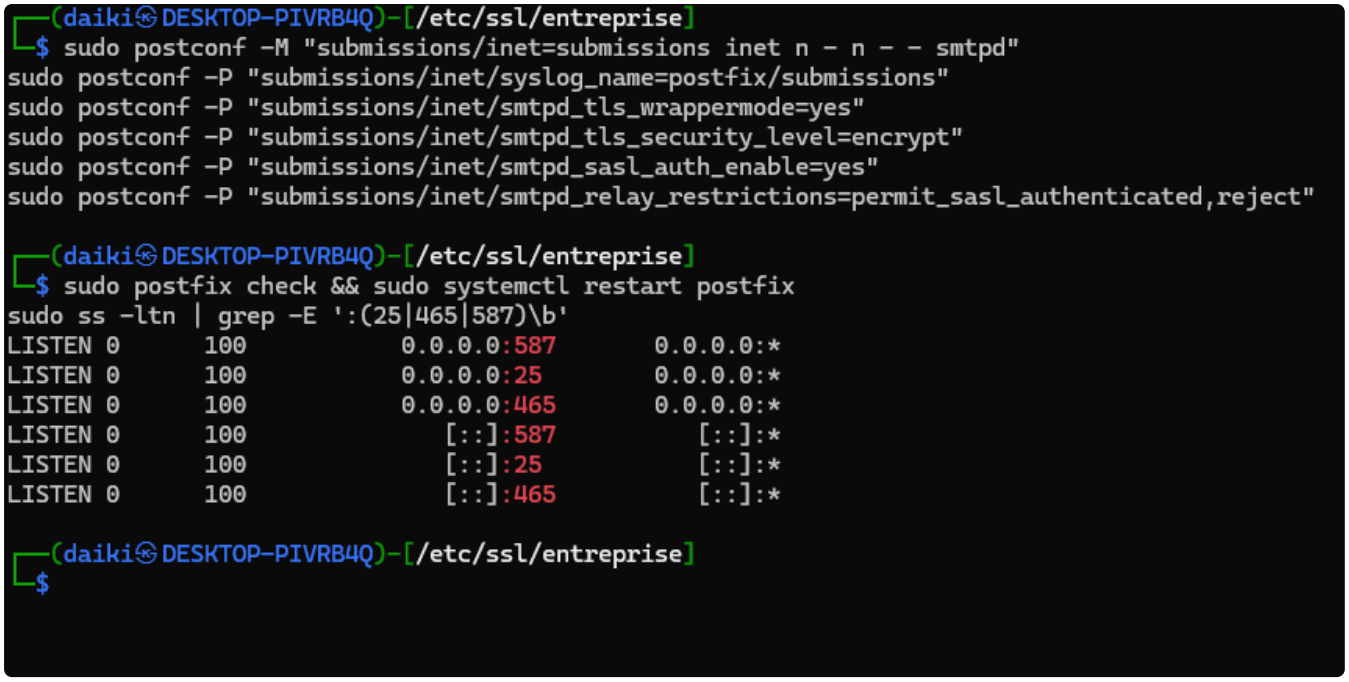


Figure 3. Activation du port 465 (submissions) et contrôle des ports à l'écoute avec ss.

1.3 Dovecot 2.4 : IMAP chiffré sur 993, port 143 fermé

Ajoutez à la fin de `/etc/dovecot/conf.d/99-entreprise.conf` (si `ssl = required` y figure déjà, ne le répétez pas) :

```
ssl = required
ssl_server_cert_file = /etc/ssl/entreprise/mail.crt
ssl_server_key_file = /etc/ssl/entreprise/mail.key
ssl_min_protocol = TLSv1.2

service imap-login {
  inet_listener imap {
    port = 0
  }
  inet_listener imaps {
    port = 993
    ssl = yes
  }
}
```

Puis `sudo doveconf -n > /dev/null && echo OK` et `sudo systemctl restart dovecot`. Relisez la sortie du redémarrage : une erreur de frappe dans le nom du service laisse l'ancienne configuration active.

```
┌──(daiki㉿DESKTOP-PIVRB4Q)-[/etc/ssl/entreprise]
└─$ sudo cp /etc/dovecot/conf.d/99-entreprise.conf /etc/dovecot/conf.d/99-entreprise.conf.bak
grep -n "ssl\|imap-login" /etc/dovecot/conf.d/99-entreprise.conf

┌──(daiki㉿DESKTOP-PIVRB4Q)-[/etc/ssl/entreprise]
└─$ sudo tee -a /etc/dovecot/conf.d/99-entreprise.conf > /dev/null <<'EOF'

ssl_server_cert_file = /etc/ssl/entreprise/mail.crt
ssl_server_key_file = /etc/ssl/entreprise/mail.key
ssl_min_protocol = TLSv1.2

service imap-login {
  inet_listener imap {
    port = 0
  }
  inet_listener imaps {
    port = 993
    ssl = yes
  }
}
EOF

┌──(daiki㉿DESKTOP-PIVRB4Q)-[/etc/ssl/entreprise]
└─$ sudo doveconf -n > /dev/null && echo OK
OK

┌──(daiki㉿DESKTOP-PIVRB4Q)-[/etc/ssl/entreprise]
└─$
```

Figure 4. Configuration TLS de Dovecot 2.4 ; doveconf ne signale aucune erreur.

```
┌──(daiki㉿DESKTOP-PIVRB4Q)-[~]
└─$ sudo systemctl restart dovecot
sudo ss -ltn | grep -E ':(143|993)\b'
[sudo] password for daiki:
LISTEN 0      100          0.0.0.0:993       0.0.0.0:*
LISTEN 0      100             [::]:993          [::]:*

┌──(daiki㉿DESKTOP-PIVRB4Q)-[~]
└─$ openssl s_client -connect mail.entreprise.local:993 -CAfile /etc/ssl/entreprise/ca.crt -brief </dev/null
Connecting to 127.0.0.1
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN=mail.entreprise.local
Hash used: SHA256
Signature type: rsa_pss_rsae_sha256
Verification: OK
Negotiated TLS1.3 group: X25519MLKEM768
DONE

┌──(daiki㉿DESKTOP-PIVRB4Q)-[~]
└─$
```

Figure 5. Port 993 à l'écoute et test openssl s_client : Verification: OK.

1.4 Roundcube : vérifier le certificat au lieu de l'ignorer

Dans `/etc/roundcube/config.inc.php`, remplacez les lignes `imap_host`, `smtp_host` et `verify_peer` ajoutées à l'étape 8 du guide :

```
$config['imap_host'] = 'ssl://mail.entreprise.local:993';
$config['smtp_host'] = 'tls://mail.entreprise.local:587';
$o = array('ssl' => array('verify_peer' => true, 'verify_peer_name' => true,
  'cafile' => '/etc/ssl/entreprise/ca.crt'));
$config['imap_conn_options'] = $o;
$config['smtp_conn_options'] = $o;
```

Le nom `mail.entreprise.local` doit résoudre vers `127.0.0.1` (fichier `hosts`, étape 2 du guide) : c'est lui, et non `127.0.0.1`, que le certificat garantit. Le fichier `ca.crt` doit être lisible par l'utilisateur d'Apache (`www-data`).

```
┌──(daiki㉿DESKTOP-PIVRB4Q)-[~]
└─$ sudo cp /etc/roundcube/config.inc.php /etc/roundcube/config.inc.php.bak
grep -n "imap_host\|smtp_host\|verify_peer\|conn_options\|\$o " /etc/roundcube/config.inc.php
grep: /etc/roundcube/config.inc.php: Permission denied

┌──(daiki㉿DESKTOP-PIVRB4Q)-[~]
└─$ getent hosts mail.entreprise.local
127.0.0.1       mail.entreprise.local

┌──(daiki㉿DESKTOP-PIVRB4Q)-[~]
└─$ sudo tee -a /etc/roundcube/config.inc.php > /dev/null <<'EOF'

// --- TLS verifie (rapport securisation, etape 1.4) ---
$config['imap_host'] = 'ssl://mail.entreprise.local:993';
$config['smtp_host'] = 'tls://mail.entreprise.local:587';
$o = array('ssl' => array(
    'verify_peer' => true,
    'verify_peer_name' => true,
    'cafile' => '/etc/ssl/entreprise/ca.crt',
));
$config['imap_conn_options'] = $o;
$config['smtp_conn_options'] = $o;
EOF

┌──(daiki㉿DESKTOP-PIVRB4Q)-[~]
└─$ sudo php -l /etc/roundcube/config.inc.php
No syntax errors detected in /etc/roundcube/config.inc.php

┌──(daiki㉿DESKTOP-PIVRB4Q)-[~]
```

Figure 6. Configuration TLS de Roundcube ; la syntaxe PHP est validée (php -l).

1.5 En production

Remplacez l'AC interne par Let's Encrypt : `sudo certbot certonly --apache -d mail.entreprise.sn`, puis pointez Postfix et Dovecot vers `/etc/letsencrypt/live/mail.entreprise.sn/fullchain.pem` et `privkey.pem`. Un script dans `/etc/letsencrypt/renewal-hooks/deploy/` doit recharger `postfix`, `dovecot` et `apache2` après chaque renouvellement. Retirez alors la ligne `cafile` de Roundcube.

2. SPF : autoriser les serveurs à envoyer pour le domaine

SPF est un enregistrement DNS qui liste les machines autorisées à envoyer du courrier pour `entreprise.sn`. Le serveur qui reçoit un message compare l'adresse IP de l'expéditeur à cette liste. SPF ne se configure donc pas dans Postfix pour l'envoi : il se publie dans le DNS, et c'est la vérification à la réception qui est faite par Rspamd (section 3).

2.1 L'enregistrement

Nom	Type	Valeur
`entreprise.sn`	TXT	`v=spf1 mx -all`

`mx` autorise les serveurs désignés par l'enregistrement MX du domaine, ici `mail.entreprise.sn`. `-all` demande de rejeter tout autre expéditeur. Si l'entreprise envoie aussi depuis un service externe (newsletter, outil de facturation), ajoutez son `include:` avant `-all`. En cas de doute, commencez par `~all` (échec souple) pendant deux semaines, puis passez à `-all`.

Un seul enregistrement SPF par domaine : deux enregistrements `v=spf1` rendent SPF invalide. Il est limité à 10 recherches DNS.

2.2 Laboratoire : un petit DNS local

Le domaine `entreprise.sn` n'est pas à vous en laboratoire : on le simule avec `dnsmasq` sur le port 5353 de la machine, que Rspamd interrogera à la place du DNS public.

```
sudo apt install -y dnsmasq dnsutils
sudo tee /etc/dnsmasq.d/entreprise-lab.conf > /dev/null <<'EOF'
port=5353
listen-address=127.0.0.1
bind-interfaces
no-resolv
address=/mail.entreprise.sn/127.0.0.1
mx-host=entreprise.sn,mail.entreprise.sn,10
txt-record=entreprise.sn,"v=spf1 mx ip4:127.0.0.1 -all"
EOF
sudo systemctl restart dnsmasq
dig @127.0.0.1 -p 5353 TXT entreprise.sn +short
```

La dernière commande doit afficher la ligne `v=spf1 ...`. Le port 5353 évite tout conflit avec le résolveur de WSL. Les enregistrements DKIM et DMARC seront ajoutés à ce même fichier aux sections 3 et 4. La mention `ip4:127.0.0.1` n'existe que pour la

maquette : en production, `mx` suffit.

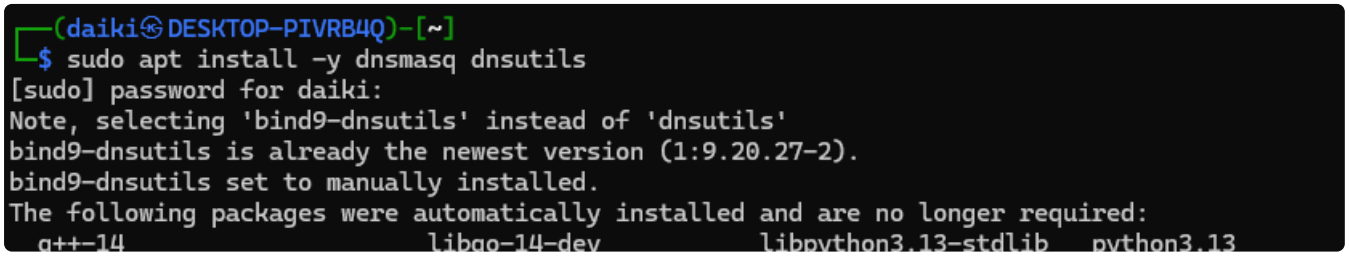


Figure 7. Installation de dnsmasq et des outils DNS (dnsutils).



Figure 8. dnsmasq 2.93 : test de syntaxe réussi puis redémarrage du service.

2.3 Production

Publiez l'enregistrement dans la zone DNS du domaine, chez le registraire ou l'hébergeur. Complétez par deux enregistrements qui ne sont pas du SPF mais conditionnent la réception : l'enregistrement MX et le DNS inverse (PTR) de l'adresse IP du serveur, à demander à l'hébergeur.

3. DKIM : signer les messages

DKIM ajoute à chaque message sortant une signature calculée avec une clé privée qui reste sur le serveur. Le destinataire récupère la clé publique dans le DNS et vérifie que le message n'a pas été modifié et qu'il vient bien du domaine. C'est Rspamd qui signe les messages sortants et qui vérifie SPF, DKIM et DMARC des messages entrants ; Postfix lui transmet chaque message par le mécanisme « milter ».

3.1 Installer Rspamd et le relier à Postfix

```
sudo apt install -y rspamd redis-server
sudo systemctl enable --now redis-server rspamd
sudo postconf -e "smtpd_milters = inet:localhost:11332"
sudo postconf -e "non_smtpd_milters = inet:localhost:11332"
sudo postconf -e "milter_protocol = 6"
sudo postconf -e "milter_default_action = accept"
sudo postconf -e "milter_mail_macros = i {mail_addr} {client_addr}
{client_name} {auth_authen}"
sudo postfix check && sudo systemctl restart postfix
```

`milter_mail_macros` permet à Rspamd de savoir quel compte est authentifié. Avec `milter_default_action = accept`, une panne de Rspamd laisse passer le courrier sans signature ni filtre ; remplacez par `tempfail` si la sécurité prime sur la disponibilité.

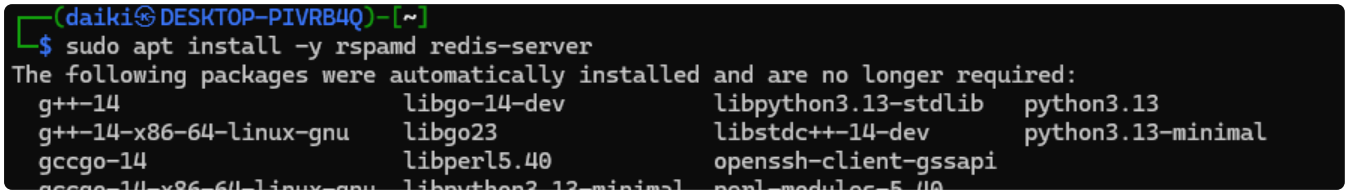


Figure 9. Installation de Rspamd et de Redis.

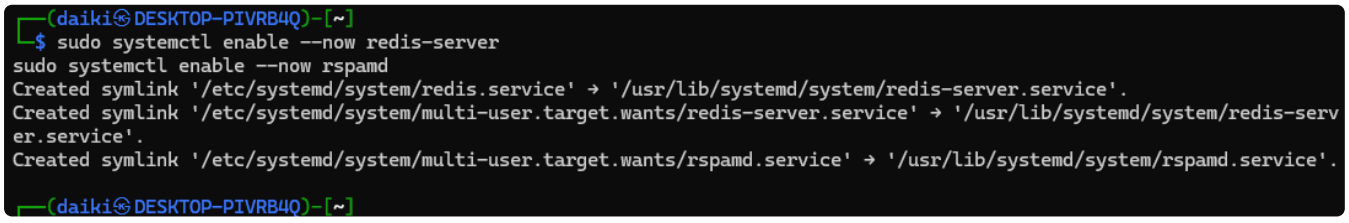


Figure 10. Activation des services redis-server et rspamd.

3.2 Générer la clé

Le sélecteur (`mail`) est le nom de la clé ; il permet d'en changer plus tard sans casser les anciens messages.

```
sudo mkdir -p /var/lib/rspamd/dkim
sudo rspamadm dkim_keygen -s mail -b 2048 -d entreprise.sn -k
/var/lib/rspamd/dkim/entreprise.sn.mail.key | sudo tee /root/dkim-
entreprise.sn.txt
sudo chown _rspamd:_rspamd /var/lib/rspamd/dkim/entreprise.sn.mail.key
sudo chmod 440 /var/lib/rspamd/dkim/entreprise.sn.mail.key
```

La commande affiche l'enregistrement TXT public à publier (copié dans `/root/dkim-entreprise.sn.txt`). La clé privée `.key` ne quitte jamais le serveur et ne se partage pas.

3.3 Activer la signature

```
sudo tee /etc/rspamd/local.d/dkim_signing.conf > /dev/null <<'EOF'
enabled = true;
sign_authenticated = true;
sign_local = true;
use_domain = "header";
allow_username_mismatch = true;
domain {
  entreprise.sn {
    selector = "mail";
    path = "/var/lib/rspamd/dkim/entreprise.sn.mail.key";
  }
}
EOF
```

3.4 Publier la clé publique

Nom	Type	Valeur
`mail._domainkey.entreprise.sn`	TXT	`v=DKIM1; k=rsa; p=` suivi de la clé publique en base64

Laboratoire. Une clé de 2048 bits dépasse les 255 caractères que tient une chaîne TXT : on la découpe en morceaux de 200 caractères, que le résolveur recolle. Le script ci-dessous extrait la clé publique et écrit la ligne dnsmasq sans recopie manuelle, puis indique à Rspamd d'utiliser ce DNS local.

```
P=$(sudo openssl rsa -in /var/lib/rspamd/dkim/entreprise.sn.mail.key -pubout
-outform der 2>/dev/null | base64 -w0)
CH=$(echo -n "v=DKIM1; k=rsa; p=$P" | fold -w 200 | sed 's/.*/"&"/' | paste -
sd, -)
echo "txt-record=mail._domainkey.entreprise.sn,$CH" | sudo tee -a
/etc/dnsmasq.d/entreprise-lab.conf > /dev/null
sudo systemctl restart dnsmasq
dig @127.0.0.1 -p 5353 TXT mail._domainkey.entreprise.sn +short

sudo tee /etc/rspamd/local.d/options.inc > /dev/null <<'EOF'
dns {
  nameserver = ["127.0.0.1:5353"];
}
EOF
sudo systemctl restart rspamd
```

Production. Supprimez `options.inc` (Rspamd utilisera le DNS du système) et publiez l'enregistrement chez l'hébergeur DNS, tel que l'affiche `/root/dkim-entreprise.sn.txt`.

3.5 Vérifier la signature

Envoyez un message (test `swaks` de l'étape 10 du guide, de `junior@entreprise.sn` vers `rh@entreprise.sn`), puis cherchez l'en-tête dans la boîte de destination :

```
sudo grep -rl "DKIM-Signature" /var/vmail/entreprise.sn/rh/
F=$(sudo grep -rl "DKIM-Signature" /var/vmail/entreprise.sn/rh/ | tail -n 1)
sudo grep -A3 "^DKIM-Signature" "$F"
sudo cat "$F" | rspamc
```

L'en-tête doit contenir `d=entreprise.sn; s=mail`. La sortie de `rspamc` doit citer `R_DKIM_ALLOW` : la signature est valide et la clé trouvée dans le DNS local. Si ce symbole manque, c'est le plus souvent que Rspamd ne joint pas le DNS local (vérifiez `options.inc`) ou que la clé recopiée est incomplète (relancez le script, il remplace la ligne en double : supprimez d'abord l'ancienne ligne `txt-record=mail._domainkey...` du fichier dnsmasq).

Rspamd ne vérifie ni SPF ni DKIM pour les connexions venant de `127.0.0.1` (réseaux locaux exemptés par défaut). Pour la maquette, cela ne gêne pas la signature des messages sortants ; en revanche, les contrôles SPF et DMARC à l'arrivée ne se voient réellement qu'avec un expéditeur extérieur, donc en production.

4. DMARC : définir le sort des messages non authentifiés

DMARC relie SPF et DKIM à l'adresse visible dans le champ « From » : un message passe si SPF ou DKIM réussit **et** si le domaine authentifié correspond au domaine du « From » (c'est l'alignement). La politique publiée dit au destinataire quoi faire des messages qui échouent, et où envoyer les rapports.

4.1 L'enregistrement

Nom	Type	Valeur
`_dmarc.entreprise.sn`	TXT	`v=DMARC1; p=none; rua=mailto:admin@entreprise.sn`

Politique	Effet sur un message qui échoue	Quand l'utiliser
`p=none`	Aucun : on observe seulement	Les deux premières semaines
`p=quarantine`	Classé en spam	Après lecture des rapports, d'abord avec `pct=25`
`p=reject`	Refusé	Quand tous les expéditeurs légitimes passent

Passez de l'une à l'autre seulement après avoir lu les rapports envoyés à l'adresse `rua` : ils révèlent les expéditeurs légitimes oubliés (outil externe, imprimante, site web). Un passage direct à `reject` fait perdre du courrier.

4.2 Laboratoire

```
echo 'txt-record=_dmarc.entreprise.sn,"v=DMARC1; p=none;
rua=mailto:admin@entreprise.sn"' | sudo tee -a /etc/dnsmasq.d/entreprise-
lab.conf > /dev/null
sudo systemctl restart dnsmasq
dig @127.0.0.1 -p 5353 TXT _dmarc.entreprise.sn +short
```

Rspamd lit cette politique à l'arrivée d'un message et l'applique (module `dmarc`, actif par défaut). Comme pour SPF, le contrôle n'est réellement visible qu'avec un expéditeur extérieur : en production, envoyez un message vers une boîte chez un autre fournisseur et lisez l'en-tête `Authentication-Results`, qui doit afficher `spf=pass`, `dkim=pass` et `dmarc=pass`.

Alignement. Le domaine du « From » (`entreprise.sn`) doit être celui de la signature DKIM (`d=entreprise.sn`). C'est pourquoi la maquette signe avec `entreprise.sn` et non `entreprise.local`.

5. Fail2ban : bloquer les tentatives répétées

Fail2ban lit les journaux, repère les adresses qui échouent plusieurs fois à se connecter et les bannit temporairement au niveau du pare-feu. Il protège les ports 465, 587 et 993 contre la force brute.

5.1 Journaux d'authentification

```
echo "auth_verbose = yes" | sudo tee -a /etc/dovecot/conf.d/99-
entreprise.conf
sudo doveconf -n > /dev/null && sudo systemctl restart dovecot
sudo apt install -y fail2ban
ls /var/log/mail.log
```

Si `/var/log/mail.log` n'existe pas (pas de `rsyslog`), remplacez dans la configuration ci-dessous chaque ligne `logpath` par `backend = systemd` et installez `python3-systemd`.

5.2 Configuration

```
sudo tee /etc/fail2ban/jail.local > /dev/null <<'EOF'
[DEFAULT]
bantime = 1h
findtime = 10m
maxretry = 5
ignoreip = 127.0.0.1/8 ::1

[postfix]
enabled = true
mode = aggressive
port = smtp,465,submission
logpath = /var/log/mail.log

[postfix-sasl]
enabled = true
port = smtp,465,submission,imap,imaps
logpath = /var/log/mail.log
maxretry = 4

[dovecot]
enabled = true
port = imap,imaps,submission,465
logpath = /var/log/mail.log
EOF
sudo systemctl enable --now fail2ban
sudo systemctl restart fail2ban
sudo fail2ban-client status
sudo fail2ban-client status postfix-sasl
```

La première commande d'état doit lister les trois prisons. Vérifiez que les filtres reconnaissent vos journaux : `sudo fail2ban-regex /var/log/mail.log dovecot` doit afficher un nombre de lignes reconnues supérieur à zéro après quelques échecs de

connexion.

5.3 Particularités de WSL

- **Pare-feu.** Fail2ban agit par `iptables` ou `nftables`. Testez : `sudo fail2ban-client set postfix-sasl banip 203.0.113.9`, puis `sudo nft list ruleset | grep 203.0.113.9` (ou `sudo iptables -S | grep 203.0.113.9`). Si rien n'apparaît, le noyau WSL ne fournit pas le module : changez `banaction` dans `[DEFAULT]` (`nftables` ou `iptables-multiport`). Débannissez ensuite avec `sudo fail2ban-client set postfix-sasl unbanip 203.0.113.9`.
- **Adresse du PC Windows.** Avec la redirection de ports (étape 13 du guide), tous les clients semblent venir de la même adresse. Cinq échecs d'un seul employé peuvent alors bannir tout le réseau : ajoutez le sous-réseau de l'entreprise et l'adresse du PC Windows à `ignoreip`.

5.4 Tester

Depuis un client, lancez cinq connexions avec un mauvais mot de passe :

```
for i in 1 2 3 4 5; do swaks --to rh@entreprise.sn --from
junior@entreprise.sn --server 127.0.0.1 --port 587 --tls --auth LOGIN --auth-
user junior@entreprise.sn --auth-password 'FAUX' > /dev/null 2>&1; done
sudo fail2ban-client status postfix-sasl
```

Depuis `127.0.0.1` (exempté par `ignoreip`), aucun bannissement ne se produit : retirez temporairement `127.0.0.1/8` de `ignoreip` pour ce test, puis remettez-le.

6. Tests de validation

Contrôle	Commande	Résultat attendu
Certificat vérifié sur 993	`openssl s_client -connect mail.entreprise.local:993 -CAfile /etc/ssl/entreprise/ca.crt -brief </dev/null`	`Verification: OK`, protocole TLSv1.2 ou 1.3
Ancien TLS refusé	`openssl s_client -connect mail.entreprise.local:587 -starttls smtp -tls1_1`	Échec de la négociation

Contrôle	Commande	Résultat attendu
Port 465 en TLS	`openssl s_client -connect mail.entreprise.local:465 -CAfile /etc/ssl/entreprise/ca.crt -brief </dev/null`	`Verification: OK`
STARTTLS obligatoire sur 587	`swaks --to rh@entreprise.sn --server 127.0.0.1 --port 587 --auth LOGIN --auth-user junior@entreprise.sn --auth-password '...'` (sans `--tls`)	`530 5.7.0 Must issue a STARTTLS command first`
IMAP en clair fermé	`ss -ltn | grep -E ':(143|110)\b'`	Aucune ligne
SPF publié	`dig @127.0.0.1 -p 5353 TXT entreprise.sn +short`	`v=spf1 mx ...`
DKIM publié	`dig @127.0.0.1 -p 5353 TXT mail._domainkey.entreprise.sn +short`	`v=DKIM1; k=rsa; p=...`
DMARC publié	`dig @127.0.0.1 -p 5353 TXT _dmarc.entreprise.sn +short`	`v=DMARC1; p=none; ...`
Message signé	`sudo grep -rl "DKIM-Signature" /var/vmail/entreprise.sn/rh/`	Au moins un fichier ; `d=entreprise.sn; s=mail`
Signature valide	`sudo cat "$F" | rspamc` (voir 3.5)	Symbole `R_DKIM_ALLOW`
Fail2ban actif	`sudo fail2ban-client status`	Prisons `postfix`, `postfix-sasl`, `dovecot`
Bannissement	Section 5.4	Une adresse listée dans `Banned IP list`
Production : trois contrôles	Message envoyé vers un autre fournisseur, lecture de `Authentication-Results`	`spf=pass`, `dkim=pass`, `dmarc=pass`

La dernière ligne ne peut pas se vérifier sur la maquette : elle demande un domaine public et un serveur joignable depuis Internet.

7. Problèmes probables et ordre de mise en œuvre

Symptôme	Cause probable	Correction
Roundcube : échec de connexion après le passage à 993	Le nom `mail.entreprise.local` ne résout pas, ou `ca.crt` illisible par `www-data`	Vérifier le fichier `hosts` ; `sudo chmod 644 /etc/ssl/entreprise/ca.crt`
Postfix : `permission denied` sur la clé	L'utilisateur `postfix` n'est pas dans le groupe `ssl-cert`	`sudo adduser postfix ssl-cert`, puis `sudo systemctl restart postfix`
`doveconf` signale une erreur sur `ssl_*`	Syntaxe 2.3 utilisée sur Dovecot 2.4	Utiliser `ssl_server_cert_file`, `ssl_server_key_file`, `ssl_min_protocol`
Aucun `DKIM-Signature` dans le message	Rspamd non relié à Postfix, ou expéditeur non authentifié	`sudo postconf smtpd_milters` ; envoyer sur le port 587 avec authentification
`rspamc` n'affiche pas `R_DKIM_ALLOW`	Rspamd n'interroge pas le DNS local, ou clé publique incomplète	Contrôler `options.inc` ; refaire le script 3.4 après avoir supprimé l'ancienne ligne
`dnsmasq` ne démarre pas	Port 53 déjà utilisé par le résolveur de WSL	Vérifier que `port=5353` est bien dans `/etc/dnsmasq.d/entreprise-lab.conf`
Fail2ban ne bannit rien	Filtre qui ne reconnaît pas les journaux, ou `127.0.0.1` dans `ignoreip`	`sudo fail2ban-regex /var/log/mail.log dovecot` ; voir 5.4
Tous les employés bloqués	Adresse du PC Windows bannie (redirection de ports)	`unbanip`, puis ajouter le sous-réseau à `ignoreip`

Ordre conseillé. 1) TLS (section 1), car SPF, DKIM et DMARC n'ont d'intérêt que sur des connexions fiables. 2) Rspamd et DKIM (section 3), avec le DNS local (2.2). 3) SPF puis DMARC en `p=none`. 4) Fail2ban. 5) Les tests de la section 6, après chaque étape.

En production. Les enregistrements SPF, DKIM et DMARC se publient chez l'hébergeur DNS du domaine ; DMARC passe de `none` à `quarantine` puis `reject` en quelques semaines (section 4.1) ; Let's Encrypt remplace l'AC interne ; le serveur tourne sur Debian stable ou Ubuntu Server LTS plutôt que Kali.

8. Avancement sur la maquette (8 et 9 novembre 2026)

TLS et SPF sont validés sur la maquette. Rspamd est installé mais pas encore relié à Postfix : DKIM et DMARC restent à faire.

Mesure	État	Preuve relevée
TLS : autorité interne et certificat	Fait	`openssl verify` : `mail.crt: OK` ; `mail.key` en 640, groupe `ssl-cert`
TLS : Postfix	Fait	Postfix 3.11.6 ; ports 25, 465 et 587 en écoute ; `Verification: OK` sur 587 (TLS 1.3)
TLS : Dovecot	Fait	`doveconf -n` : OK ; port 993 en écoute, 143 fermé ; `Verification: OK` sur 993
TLS : Roundcube	Configuré, test de connexion à confirmer	`php -l` : aucune erreur ; lignes `ssl://mail.entreprise.local:993` appliquées en dernier
Let's Encrypt (1.5)	Impossible en laboratoire	Voir 8.1
SPF et DNS local	Fait (publication)	dnsmasq 2.93, syntaxe OK, enregistrements TXT, MX et A obtenus avec `dig` ; évaluation à l'arrivée à tester avec Rspamd
Rspamd et Redis	Installés	Rspamd 4.0.1 actif, proxy en écoute sur `127.0.0.1:11332`
Liaison Postfix et Rspamd (3.1)	À faire	`postconf -n` ne renvoie aucun paramètre milter
DKIM (3.2 à 3.5), DMARC (4), Fail2ban (5)	À faire	

8.1 Problèmes rencontrés

Commandes coupées au collage. Les longues commandes OpenSSL du point 1.1 ont été collées dans le terminal puis coupées en milieu de ligne : `-subj` sans valeur, `mail.csr: command not found`. Les clés étaient créées, pas les certificats. Correction : couper

chaque commande en plusieurs lignes avec `\` en fin de ligne, et exécuter une commande à la fois. Un contrôle final avec `sudo openssl verify -CAfile ca.crt mail.crt` doit répondre `OK`.

Let's Encrypt refusé (NXDOMAIN). La commande `certbot certonly --apache -d mail.entreprise.sn` échoue avec `NXDOMAIN looking up A for mail.entreprise.sn`. C'est le résultat attendu : le nom n'existe pas dans le DNS public, et Let's Encrypt doit le joindre depuis Internet. Les enregistrements publiés dans dnsmasq ne sont visibles que sur la machine. Conclusion : le point 1.5 ne s'applique qu'à un serveur de production avec un vrai domaine ; la maquette garde l'autorité interne. La création du compte certbot est sans conséquence.

Mot de passe `sudo` et `grep`. La lecture de `/etc/roundcube/config.inc.php` demande `sudo`, sans quoi `grep` répond `Permission denied`.

Mémoire. Rspamd consomme environ 500 Mo (pic à 700 Mo) sur la maquette. Avec les 4 Go alloués à WSL, c'est acceptable ; en cas de lenteur, surveiller avec `free -h`.

8.2 Prochaines actions

1. Relier Postfix à Rspamd (3.1), puis relancer `sudo postconf -n | grep milter` : les cinq paramètres doivent apparaître.
2. Générer la clé DKIM, publier la clé dans dnsmasq, activer la signature (3.2 à 3.4).
3. Envoyer un message de `junior@entreprise.sn` vers `rh@entreprise.sn` et vérifier `DKIM-Signature` et `R_DKIM_ALLOW` (3.5).
4. Publier DMARC en `p=none` (4.2), puis installer Fail2ban (5).
5. Rejouer les tests de la section 6 et noter les résultats ici.